Demo Script

Ghost Data Audit — rehearsal script for the live interview

Timer chips are cumulative elapsed time, not per-beat duration. Read the bold cue as a stage direction, then speak the line under it in your own words — it's a script to rehearse from, not to read aloud verbatim.

Numbers below are pulled from code/evaluation/evaluation_report.md and usage_report.md at final-run time: 0.812 status accuracy, 0.706 step F1, 518/518 full-run rows valid, 17 LLM calls at $0.006.

Part 1 — The 60-Second Pitch

Open with this before touching the keyboard. Memorize the shape, not the words.

0:00 **State the problem in one breath**

"Ghost Data Still Exists" asks for a deletion-audit agent over 518 requests and a 94-file multi-system evidence corpus.

0:10 **Name the architecture choice immediately**

I built it evidence-first: parse the entire shared corpus once into a flat observation log, resolve each user's full identity footprint — current and historical emails, hashes, phones — then reconstruct what each resource's state actually was at the audit timestamp.

0:28 **State the one rule that matters most**

File evidence outranks activity claims, because a claim is just a claim. Then I apply 15 deletion rules deterministically to produce an ordered finding / remediation / verification plan.

0:40 **Defend the one LLM call**

The only LLM call in the whole pipeline is request-intent classification, because that's the one genuinely open-ended, multilingual task. Everything else is deterministic Python, so a judge re-running my exact commands gets my exact output every time.

0:52 **Land the numbers and stop talking**

Final sample scores: 0.812 status accuracy, 0.706 step F1. Let that sit — don't keep talking into the silence.

Part 2 — The Visualizer Walkthrough (~90s)

Run first: python -m http.server 8765 -d tools/viz → open http://localhost:8765

0:00 **Open on the stat marquee**

"This is the same pipeline from the README, but you can watch a case move through it live." Let the KPI numbers count up before you say anything else.

0:12 **Name what the numbers are**

"These are the real numbers from my last validated run — status accuracy, step F1, sequence accuracy, the status distribution across all 518 cases, and total LLM cost."

0:28 **Click the Intent node**

"This is the only LLM call in the whole system — here's what it's for, and its exact token and cost usage." Point at the per-call ledger, not just the total.

0:45 **Click through the deterministic stages**

Evidence Ingest → Point-in-time State → Policy → Planner. "Everything after intent is deterministic Python — same input, same output, every time."

1:05 **Pick a case and hit Trace case**

"Here's one case end to end — which user it resolved to, which rule found their account, every resource with its state and evidence citation, and the final step timeline in the drawer." Let the trace animation finish before talking over it.

1:25 **Close the loop**

"Every one of these came from a real file in build/ — nothing hand-written in this view." This is the line that proves it isn't a mockup.

Part 3 — Failure Stories (pick 2, don't recite all 5)

These answer "tell me about a bug you hit" or "what went wrong." Lead with the symptom, not the fix — the fix lands harder after the symptom.

— The quiet one: NOT_APPLICABLE collapse

A planner bug silently collapsed "zero resources resolved" into NOT_APPLICABLE. It looked like correct behavior right up until I checked it against a gold set that had zero NOT_APPLICABLE cases — that mismatch is what caught it, not a crash or a stack trace.

— The scary one: the parallel-session race

A session I thought I'd closed never actually stopped — it kept running and committing in the background while a new session worked the same repo. I caught it because files were changing without my edits, found two stray background processes, paused instead of racing it, and re-verified every one of its claims before keeping any of it.

— The honest one: the reverted image merge

Screenshot parsing via a vision LLM works and is fully built, but two separate attempts to merge its output moved the sample score in opposite directions on different runs. Traced it to the vision model returning a different row count for the same cached image across separate calls. Reverted both times — a number going up once isn't evidence.

Part 4 — Hard Questions, Rehearsed Answers

Read each question cold, say the answer out loud before reading it. If it doesn't come out in 2-3 sentences, it's not rehearsed enough yet.

Why not an LLM-per-case agent instead of deterministic rules?

Defensibility and reproducibility — a judge re-running my commands should get my exact output.csv, and I need to point at the rule that produced each step, not a model's reasoning trace.

How do you know your account-resolution rule isn't just guessing?

It's a strict priority chain — direct evidence link, then catalog link, then same-day batch order, then zero-padded fallback — and every case's resolution traces to build/traces/<case_id>.json.

What happens if two files disagree about a resource's state?

Point-in-time rule: the latest observation at or before audit_at wins; file evidence outranks activity claims; an exact-timestamp tie favors PRESENT — replica drift, the safer assumption.

How do you prevent prompt injection from the evidence corpus?

Every prompt that touches corpus content treats it explicitly as data, not instructions, and every structured field the LLM returns — resource_id especially — is regex-validated before being trusted.

How would this scale to 10x the evidence volume?

The ingest pass is already a single linear scan producing a flat observation log, so it scales roughly linearly; per-case resolution is a dict lookup, not a corpus re-scan.

Why is step F1 (0.706) lower than step precision (0.807) would suggest?

Recall is the gap — some true steps are missed because the resource was never linked to the user by any parsed evidence, a coverage gap, not a planner reasoning error.

Full 10-question set and architecture walkthrough: submission/INTERVIEW_PREP.md